

Хід роботи

У середовищі розробки було підготовлено проєкт для мікроконтролера PIC18F4550 відповідно до вимог лабораторної роботи № 4.

На першому етапі було проаналізовано структуру проєкту з використанням переривань та підготовлено підпрограму ініціалізації таймера TMR0 для формування частоти дискретизації.

Для TMR0 було обрано режим роботи з переповненням та визначено значення передподільника і початкового завантаження таймера.

Далі було виконано ініціалізацію модуля АЦП: налаштовано аналоговий вхід, формат результату перетворення, тактові параметри АЦП та час вибірки.

Після цього реалізовано процедуру запуску аналого-цифрового перетворення, очікування завершення перетворення та зчитування результату з регістрів ADRESH і ADRESL.

Для відображення та контролю результатів передбачено передачу цифрових даних у термінал комп'ютера через USART, а також опрацьовано варіант із використанням переривання від модуля АЦП.

Окремо було розглянуто алгоритм перетворення 16-бітного двійкового коду в двійково-десятковий формат BCD за методом послідовних зсувів і корекції тетрад.

На завершальному етапі виконано нормування цифрового коду АЦП для переходу від сирого значення $V/2^{10}$ до фізичної величини, що дозволяє отримати більш зручне числове представлення результату.

На основі методичних вказівок сформовано структуру звіту, що відповідає логіці виконання лабораторної роботи та може бути доповнена власними фрагментами коду і скріншотами моделювання.

Фрагменти програми

```
1      list p=18f4550;
2      #include <p18f4550.inc>
3
4      cblock 20
5          levelH
6          levelL
7          d1
8          d2
9          d3
10         count
11         temp
12         temp16H
13         temp16L
14         R0
15         R1
16         R2
17         seg0
18         seg1
19         seg2
20         seg3
21         arg1H
22         arg1L
23         arg2H
24         arg2L
25         res0
26         res1
27         res2
28         res3
29     endc
30
31     org 000000
32     goto start
33
34     org 000008
35     goto HighInt
36
37     start
38         call System_Init
39         call USART_Init
40         call TMRO_Init
41
42
43         ; call case3
44         ; call case4
45         ; call case5
46
47         goto Loop
48
49     Loop
50         goto Loop
51
52     System_Init
53         bsf INTCON, PEIE
54         bsf INTCON, GIE
55
56         return
57
58     USART_Init
59         movlw 0x00
60         movwf SPBRGH
61         bcf BAUDCON, BRG16
62         bcf TXSTA, BRGH
63         movlw .38
64         movwf SPBRG
65
66         bcf TXSTA, SYNC
67         bsf RCSTA, SPEN
68
69         bcf TXSTA, TX9
70         bsf TXSTA, TXEN
71
72         return
73
74     TMRO_Init
75         movlw b'10000111'
76         movwf T0CON
77
78         bsf INTCON, TMR0IE
```

```

76      movwf T0CON
77
78      bsf INTCON,TMR0IE
79      bsf INTCON2,TMR0IP
80
81      movlw 0xFF
82      movwf TMR0H
83      clrf TMR0L
84
85      return
86
87  ADC_Init
88      bsf TRISA,RA0
89      bsf TRISA,RA1
90
91      movlw b'00001101'
92      movwf ADCON1
93      bsf ADCON0,ADON
94      movlw b'10111110'
95      movwf ADCON2
96
97      return
98
99
100  HighInt
101      call TMR0_Interrupt
102  ;    call ADC_Interrupt
103
104      retfie
105
106  TMR0_Interrupt
107      movlw 0xFF
108      movwf TMR0H
109      clrf TMR0L
110      bcf INTCON,TMR0IF
111
112      call case1
113  ;    call case2
114
115      return
116

```

```

114
115      return
116
117
118  case1
119
120  checkTX1
121      btfss TXSTA,TRMT
122      goto checkTX1
123      movlw 'O'
124      movwf TXREG
125
126  checkTX2
127      btfss TXSTA,TRMT
128      goto checkTX2
129      movlw 'k'
130      movwf TXREG
131
132  checkTX3
133      btfss TXSTA,TRMT
134      goto checkTX3
135      movlw '!'
136      movwf TXREG
137
138  checkTX4
139      btfss TXSTA,TRMT
140      goto checkTX4
141      movlw '\n'
142      movwf TXREG
143
144  checkTX5
145      btfss TXSTA,TRMT
146      goto checkTX5
147      movlw '\r'
148      movwf TXREG
149
150      return
151
152
153  case2

```

```

152
153 case2
154     call delayADC
155
156     bsf ADCON0,GO/DONE
157
158 checkADC
159     btfss ADCON0,GO/DONE
160     goto checkADC
161
162     movff ADRESH,levelH
163     movff ADRESL,levelL
164
165 checkTX6
166     btfss TXSTA,TRMT
167     goto checkTX6
168
169     movff levelL,TXREG
170
171     return
172
173
174 ADC_Interrupt
175     call delay500
176
177     movff ADRESH,levelH
178     movff ADRESL,levelL
179
180 ;     clrf seg0 ;case3
181 ;     clrf seg1
182 ;     clrf seg2
183 ;     movff levelL,seg3 ;end case3
184
185 ;     movff levelH,temp16H ;case4
186 ;     movff levelL,temp16L ;end case4
187
188 ;     movff levelH,arg1H ;case5
189 ;     movff levelL,arg1L
190 ;     movlw b'00000100'
191 ;     movwf arg2H
192
193 ;     movlw b'11100010'
194 ;     movwf arg2L
195
196 ;     call mult1616
197
198 ;     movff res2,temp16H
199 ;     movff res1,temp16L ;end case5
200
201
202 ;     call bin_to_bdc ;block for case4 and case5
203
204 ;     movlw 0xf0
205 ;     andwf R1,w
206 ;     movwf seg0
207 ;     swapf seg0,f
208
209 ;     movlw 0xf0
210 ;     andwf R1,w
211 ;     movwf seg1
212
213 ;     movlw 0xf0
214 ;     andwf R2,w
215 ;     movwf seg2
216 ;     swapf seg2,f
217
218 ;     movlw 0xf0
219 ;     andwf R2,w
220 ;     movwf seg3 ;end block for case4 and case5
221
222     call sendUSART
223
224 ;     call delay500
225
226     bcf PIR1,ADIF
227     bsf ADCON0,GO/DONE
228
229     return

```

```

227     bsf ADCON0,GO/DONE
228
229     return
230
231
232     case3
233         bcf INTCON,TMR0IE
234         bcf INTCON2,TMR0IP
235
236         bsf PIE1,ADIE
237         bsf IPRI,ADIP
238
239         bsf ADCON0,GO/DONE
240
241         return
242
243
244     case4
245         bcf INTCON,TMR0IE
246         bcf INTCON2,TMR0IP
247
248         bsf PIE1,ADIE
249         bsf IPRI,ADIP
250
251         bsf ADCON0,GO/DONE
252
253         return
254
255     case5
256         bcf INTCON,TMR0IE
257         bcf INTCON2,TMR0IP
258
259         bsf PIE1,ADIE
260         bsf IPRI,ADIP
261
262         bsf ADCON0,GO/DONE
263
264         return
265
266     multi1616

```

```

265
266     multi1616
267         movf arg1L,w
268         mulwf arg2L
269
270         movff PRODH,res1
271         movff PRODL,res0
272
273         movf arg1H,w
274         mulwf arg2H
275
276         movff PRODH,res3
277         movff PRODL,res2
278
279         movf arg1L,w
280         mulwf arg2H
281
282         movf PRODL,w
283         addwf res1,f
284         movf PRODH,w
285         addwfc res2,f
286         clrf WREG
287         addwfc res3,f
288
289         movf arg1H,w
290         mulwf arg2L
291
292         movf PRODL,w
293         addwf res1,f
294         movf PRODH,w
295         addwfc res2,f
296         clrf WREG
297         addwfc res3,f
298
299         return
300
301     sendUSART
302
303     checkTX13
304         btfss TXSTA,TRMT

```

```

303 checkTX13
304     btfss TXSTA,TRMT
305     goto checkTX13
306
307     movlw '\f'
308     movwf TXREG
309
310 checkTX7
311     btfss TXSTA,TRMT
312     goto checkTX7
313
314     movlw 0x30
315     addwf seg0,w
316     movwf TXREG
317
318 checkTX8
319     btfss TXSTA,TRMT
320     goto checkTX8
321
322     movlw 0x30
323     addwf seg1,w
324     movwf TXREG
325
326 checkTX9
327     btfss TXSTA,TRMT
328     goto checkTX9
329
330     movlw 0x30
331     addwf seg2,w
332     movwf TXREG
333
334 checkTX10
335     btfss TXSTA,TRMT
336     goto checkTX10
337
338     movlw 0x30
339     addwf seg3,w
340     movwf TXREG
341
342 checkTX11

```

```

340     movlw TXREG
341
342 checkTX11
343     btfss TXSTA,TRMT
344     goto checkTX11
345
346     movlw '\n'
347     movwf TXREG
348
349 checkTX12
350     btfss TXSTA,TRMT
351     goto checkTX12
352
353     movlw '\r'
354     movwf TXREG
355
356     return
357
358 bin_to_bdc
359     bcf STATUS,C
360     movlw .16
361     movwf count
362     clrf R0
363     clrf R1
364     clrf R2
365 loop16a2
366     rlcw temp16L,f
367     rlcw temp16H,f
368     rlcw R2,f
369     rlcw R1,f
370     rlcw R0,f
371     decfsz count,f
372     goto AdjDec2
373     return
374 AdjDec2
375     movlw R2
376     clrf FSR0H
377     movwf FSR0L
378     call AdjBCD2
379     movlw R1
380     movwf FSR0L

```

```

378     call AdjBCD2
379     movlw R1
380     movwf FSR0L
381     call AdjBCD2
382     movlw R0
383     movwf FSR0L
384     call AdjBCD2
385     goto loop16a2
386 AdjBCD2
387     movlw 3
388     addwf INDF0,w
389     movwf temp
390     btfsc temp,3
391     movwf INDF0
392     movlw 30
393     addwf INDF0,w
394     movwf temp
395     btfsc temp,7
396     movwf INDF0
397     return
398
399
400 delayADC
401     movlw d'99'
402     movwf d1
403
404 delayADC_inner
405     decfsnz d1
406     return
407     goto delayADC_inner
408
409
410 delay500
411     movlw d'18'
412     movwf d3
413 delay500_outer2
414     movlw d'255'
415     movwf d2
416 delay500_outer1
417     movlw d'255'

```

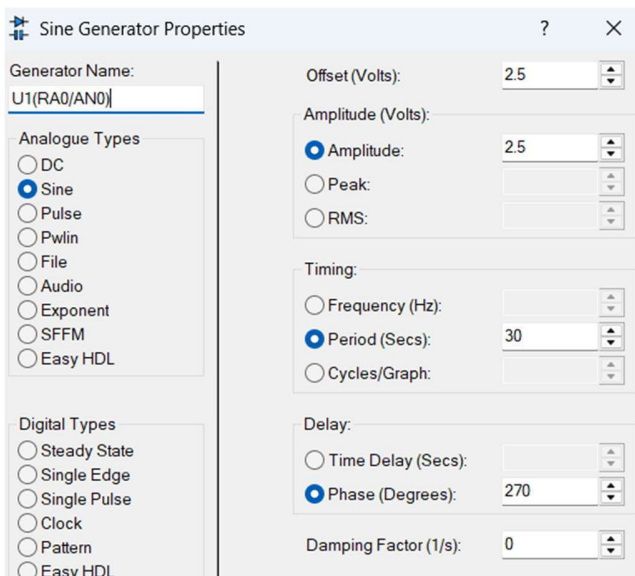
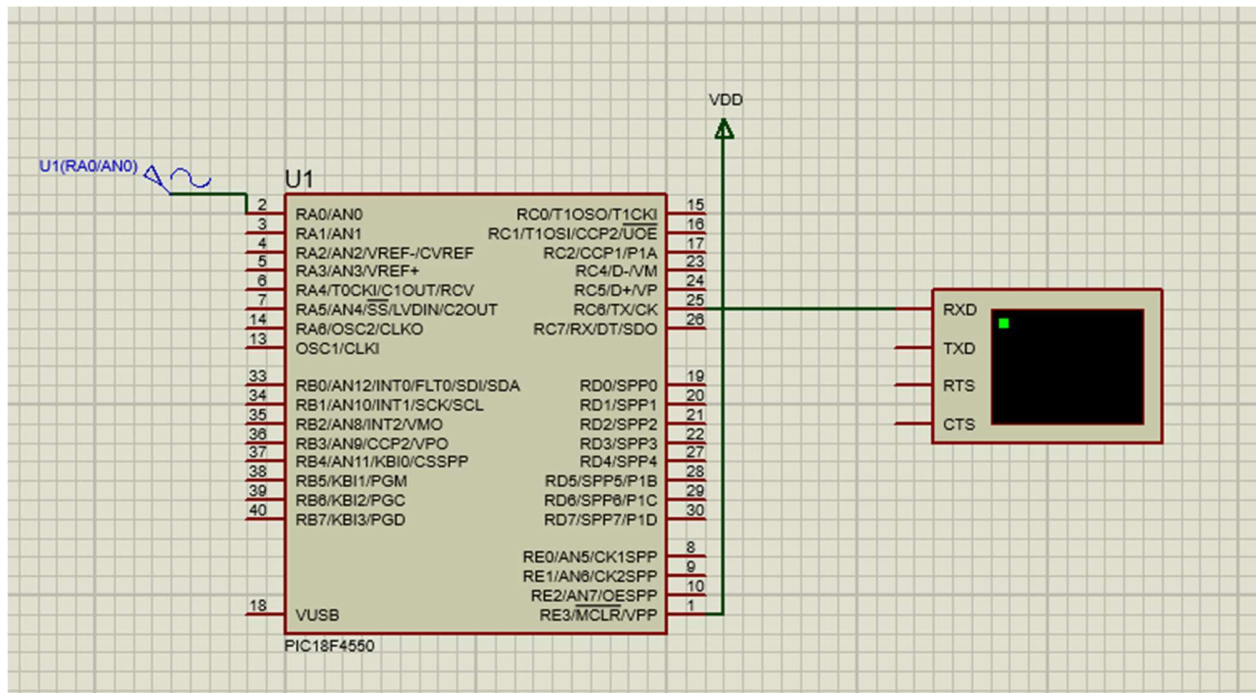
```

407     goto delayADC_inner
408
409
410 delay500
411     movlw d'18'
412     movwf d3
413 delay500_outer2
414     movlw d'255'
415     movwf d2
416 delay500_outer1
417     movlw d'255'
418     movwf d1
419 delay500_inner
420     nop
421     nop
422     decfsz d1
423     goto delay500_inner
424
425     decfsz d2
426     goto delay500_outer1
427
428     decfsz d3
429     goto delay500_outer2
430
431     return
432
433 end

```


Схема підключення та результат моделювання

У моделі використовується мікроконтролер PIC18F4550, аналоговий вхід якого підключається до джерела змінної напруги. Таймер TMR0 формує період дискретизації, модуль АЦП виконує перетворення аналогового сигналу в цифровий код, а модуль USART забезпечує передавання результатів до віртуального термінала.



Було використано саме змінне джерело напруги замість потенціометра через несправність потенціометра у версії Proteus, встановленій на моєму ПК (при запуску симуляції регулювати подільник опору неможливо).